

The Evolutionary Thought Process: Diagnosing and Re-Architecting AquaAssist Beyond Standard RAG

1. The Testing Reality Check — What Went Wrong

When AquaAssist was tested on real-world inputs (like broken English, mix-and-match Telugu, missing data, and tricky contradictions), we found major cracks:

- **The AI was twisting words:** If a farmer said, *"My shrimp are swimming near the surface,"* our tiny Qwen-0.5B model would occasionally panic and rewrite it as, *"My shrimp are **not** swimming near the surface"*—literally reversing the meaning. Or, if a farmer said oxygen *"might be low,"* the AI turned it into a strict command: *"Check the oxygen!"*
- **It couldn't read numbers:** Even when a farmer typed exact numbers like *Dissolved Oxygen = 2 mg/L*, our AI verification model (DeBERTa) gave it a score of *0.0006* (meaning "this makes no sense"). Why? Because to an AI, numbers are just alphabet soup; it had no idea that *2* meant a life-or-death emergency compared to *6*.
- **It ignored arguments:** If a farmer typed two conflicting numbers (e.g., *"One test says oxygen is 7, another says 2"*), the system noticed the conflict during extraction, but then completely ignored it and gave a generic answer anyway.
- **It spoke robot:** When something went wrong, the app spat out developer jargon and system codes like *CLARIFY*, *ABSTAIN*, or raw JSON traces right onto the screen. A rural farmer standing by a pond couldn't care less about pipeline logs; they need a human voice telling them what to do.

Why Our First Ideas Failed:

Our initial reaction was to apply quick band-aids: writing rigid Regex patterns to catch specific numbers, writing longer prompt instructions for the 0.5B model, or increasing the word-window limit for the NLI cross-encoder.

However, deep reflection revealed why these patches fail generalized engineering:

- **Regex is Brittle:** Hardcoding rules for specific test cases breaks the moment a farmer changes their phrasing, violating the core goal of building an intelligent assistant.
- **SLM Limitations:** A 0.5B parameter model cannot reliably maintain multiple simultaneous negative constraints (e.g., "don't reverse meaning, don't add causes, preserve uncertainty") via prompt engineering alone; prompt leaks and semantic drift are inevitable.
- **NLI Token Blindness:** Increasing token window sizes only stops sentences from cutting

off mid-way; it doesn't teach an entailment model what a dangerous numerical threshold actually means.

We realized we were treating symptoms rather than curing the architectural root cause. To build a true zero-hallucination field assistant, we needed to abandon naive RAG paradigms entirely.

The Breakthrough!

We decided to step back and fix the root causes:

- Instead of letting a small AI free-text rewrite sentences, we used **smart grammar rules (triplets) to lock down the exact facts**.
- Instead of expecting an AI to magically understand numbers, we **changed how we store documents in our database so the rules and numbers are explicitly laid out**.
- Instead of showing code jargon on the screen, we built **a translation layer that converts system states into warm, simple, helpful advice in Telugu or English**.

Detailed Step-by-Step Flow of the New AquaAssist

Here is how a message travels through the improved system from start to finish:

Step 1: The Input & Language Normalization

- **What happens:** The farmer speaks or types their problem (in Telugu, English, or a mix). If it's voice, local Whisper turns it into text.
- **The fix:** The system secures important technical words (**shrimp, oxygen, pH**) so they don't get lost or mistranslated during local processing.

Step 2: The Grammar & Triplet Parser (Fixing the Extractor)

- **What happens:** Instead of asking an unreliable AI to rewrite the sentence, a precise grammar parser breaks the sentence down into clean, factual blocks (called triplets)—like mapping out **[Subject] -> [Action] -> [Value]**.
- **Why it matters:** This locks in the exact meaning, keeps uncertainty phrases intact ("oxygen *might* be low"), and stops the AI from reversing words or making things up.

Step 3: Strict Quality Filtering (Stopping Junk Data)

- **What happens:** The clean sentence searches our local database (ChromaDB) using our embedding model (BGE-M3).
- **The fix:** We set a strict quality filter (similarity threshold at **0.80**). If the database tries to

return useless, tiny fragments like just the word "*shrimp*" or "*water*", the system throws them in the trash instantly.

Step 4: Propositional Knowledge Match (Fixing the Number Blind Spot)

- **What happens:** When our database was originally built, we didn't just paste raw paragraphs. We translated the ICAR-CIBA agricultural guidelines into clear, conditional rules (propositions) that explicitly state what specific numbers mean (e.g., "*Oxygen levels between 0 and 3 mg/L mean severe danger*").
- **Why it matters:** This bridges the gap so our AI verification model can actually understand numerical risk boundaries without getting confused.

Step 5: The NLI Verification Gate (The Safety Auditor)

- **What happens:** Our fine-tuned DeBERTa model compares the farmer's problem against the retrieved guidelines. Because the guidelines are structured properly, the AI can mathematically verify whether the evidence truly supports the claim.

Step 6: The Three-Way Decision Router

- **What happens:** Based on the AI's math, the system sorts the situation into one of three clean paths:
 - **ANSWER:** The evidence is strong and safe; we have a verified solution.
 - **CLARIFY:** We need a little more info (e.g., missing a parameter).
 - **CONFLICT / ABSTAIN:** The farmer gave conflicting numbers (like 7 vs 2), or safety rules are triggered. It stops everything to prevent giving wrong advice.

Step 7: The Conversational Persona UX Layer (The Farmer-Friendly Voice)

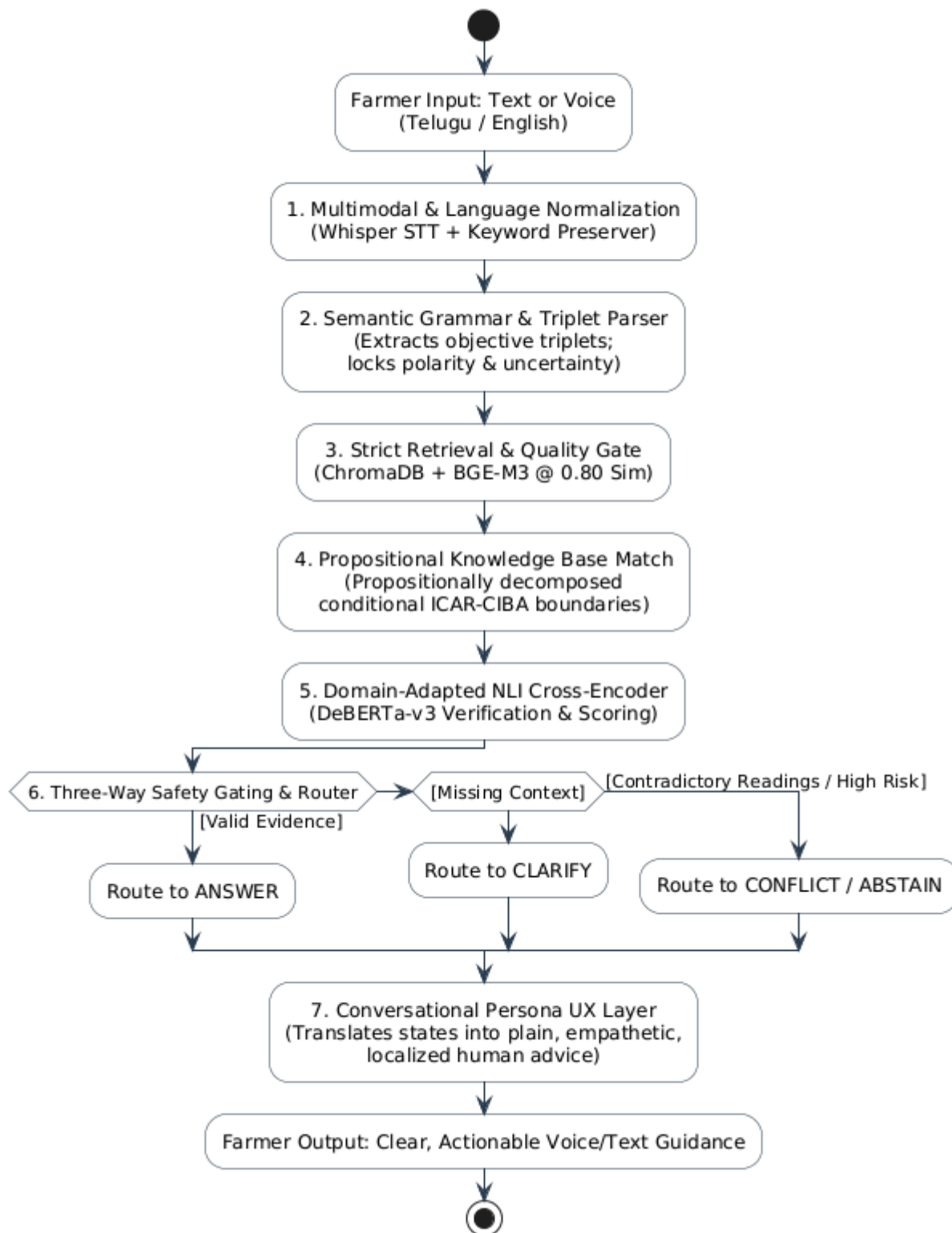
- **What happens:** The backend code packages its decision, stripping out all developer logs, JSON, and ugly state codes (**CLARIFY** / **ABSTAIN**).
- **The Result:** A translation template turns that data into warm, plain, spoken or written advice in the farmer's preferred language.
 - *Example if there's a conflict: "Hold on—you mentioned two different oxygen numbers (7 and 2). That doesn't match up. Could you check your test kit and give me one fresh reading?"*
 - *Example if it's an answer: "Your shrimp coming to the surface with an oxygen level of 2 mg/L points to low oxygen stress. Turn on your aerators immediately."*

How This Differs from Standard/Existing Pipelines

Architectural Feature	Standard / Conventional RAG Pipelines	The Modified AquaAssist Pipeline	Why AquaAssist Wins
Ingestion & Storage	Stores raw, unstructured text chunks as-is from documents.	Propositional Decomposition: Text is converted into explicit conditional rules and numerical boundary statements during ingestion.	Enables text-based NLI models to reason about quantitative metrics (like dissolved oxygen levels) without failing on blind spots.
Extraction Method	Relies on naive LLM prompting or free-text generation, causing semantic drifting and polarity reversals.	Grammar & Triplet Parsing: Extracts strict semantic structures, locking in original intent, polarity, and conditional uncertainty.	Eliminates prompt-leakage, hallucinated symptoms, and inverted meanings (e.g., turning "not eating" into "eating").
Verification & Safety	None. Passes retrieved chunks directly to a generative LLM and hopes it doesn't hallucinate.	Domain-Adapted NLI Gating: Mathematically verifies premise-hypothesis entailment via a fine-tuned cross-encoder before any answer is built.	Acts as an unyielding audit barrier; if evidence doesn't support the claim, it blocks the answer entirely.
Execution Environment	Cloud-dependent (OpenAI, Pinecone, LangChain cloud services).	100% Local-First Edge Architecture: Runs on Apple Silicon (MPS), CUDA, or CPU with hybrid sync for rural offline stability.	Usable in remote, signal-dead fishponds without cloud API keys, data privacy risks, or internet dependency.
User Experience (UI)	Displays raw chat text, system errors, or generic fallback strings ("No action found").	Conversational Persona Layer: Translates internal logic into plain, localized, empathetic human advice (Telugu/English).	Prevents farmers from being overwhelmed by developer logs or tech jargon; delivers instant, actionable clarity.

SYSTEM ARCHITECTURE FLOW IN THE FORM OF ACTIVITY DIAGRAM:

AquaAssist - Modified System Architecture Flow



How We're Gonna Achieve This:

Phase 1: Semantic Extraction & Triplet Parser (Fixing the Extractor)

- **Objective:** Replace unconstrained free-text SLM generation with a deterministic dependency/triplet parser to lock down polarity, eliminate hallucinations, and preserve uncertainty.
- **Key Actions:**
 - Build a hybrid extraction module that catches key parameters (e.g., species, symptoms, conditions) using structured grammars rather than loose prompts.
 - Enforce strict validation rules so phrases like "oxygen may be low" retain their conditional doubt rather than being converted into commands or inverted into negatives.
- **Resolves Test Issues:** Test Case 1, 2, 5, 7, and 8 (Extraction polarity reversals, loss of uncertainty, prompt contamination).

Phase 2: Propositional Knowledge Base Decomposition (Fixing NLI Numerical Blindness)

- **Objective:** Re-engineer the document ingestion pipeline to solve the core reason why DeBERTa fails on numbers (2 mg/L vs 6 mg/L).
- **Key Actions:**
 - Audit and re-chunk the ICAR-CIBA agricultural manuals stored in `chroma_db`.
 - Convert raw paragraphs into **explicit conditional propositions** (e.g., structuring chunks to explicitly declare numerical risk boundaries like "Dissolved oxygen between 0–3 mg/L entails severe stress").
- **Resolves Test Issues:** Test Case 3 and 4 (NLI entailment score drops on numerical data).

Phase 3: Retrieval Quality Filtering & Threshold Locking

- **Objective:** Stop fragmented micro-chunks and irrelevant text from poisoning the downstream NLI verification stage.
- **Key Actions:**
 - Lock the BGE-M3 vector similarity cutoff strictly to **0.80**.
 - Implement minimum character/word-length pre-filters to automatically discard junk chunks (like standalone words such as "shrimp." or "water.") before verification.
- **Resolves Test Issues:** Test Case 1 and 2 (Irrelevant retrieval chunks pulling down scores).

Phase 4: Hardened Conflict Detection & Three-Way Safety Gating

- **Objective:** Ensure contradictory sensor inputs or safety-critical requests never result in blind, ungrounded answers.
- **Key Actions:**
 - Upgrade the pre-flight check (`detect_conflicting_measurements`) to catch multi-value parameter clashes (e.g., `$7\text{ mg/L}$` vs `$2\text{ mg/L}$`) before execution.
 - Lock the decision router to force a strict **CONFLICT/ABSTAIN** or **CLARIFY** path when contradictions or low confidence are detected.

- **Resolves Test Issues:** Test Case 4, 6, and 7 (Ignoring sensor contradictions and missing clarification flows).

Phase 5: Conversational Persona UX Layer (Farmer-Friendly Rendering)

- **Objective:** Completely strip away internal developer codes (CLARIFY, ABSTAIN), JSON traces, and technical jargon from the user-facing interface.
- **Key Actions:**
 - Build a localized translation and persona template engine in Streamlit.
 - Map every system decision state to warm, plain, empathetic human advice in Telugu or English (e.g., turning a routing state into a direct, helpful recommendation or a polite request for a re-measurement).
- **Resolves Test Issues:** Test Case 8, 10, and general UI/UX friction (Exposing developer logic and unhelpful fallback strings to rural users).